

Тема 6. Списъци, кортежи и речници

Списъци, кортежи и речници

Списък

За да работи с набори от данни, Python предоставя вградени типове като списъци, кортежи и речници. Списък е тип данни, който съхранява набор или последователност от елементи. Много езици за програмиране имат подобна структура от данни, наречена масив.

Създаване на списък

За създаване на списък се използват квадратни скоби [], вътре в които се изброяват елементите на списъка, разделени със запетаи. Например, нека да се дефинира списък с числа:

```
numbers = [1, 2, 3, 4, 5]
```

По същия начин може да се дефинира списък с други типове данни, като списък от низове:

```
people = ["Tom", "Sam", "Bob"]
```

Може също да се използва функцията за конструктор list(), за да се създаде списък:

```
numbers1 = []
```

```
numbers2 = list()
```

И двете дефиниции на списъци са сходни - те създават празен списък. Списъкът може да съдържа обекти различен тип т.е. Може да се поставят низове, числа, обекти и от други типове данни в един и същ:

```
objects = [1, 2.6, "Hello", True]
```

За да се проверят елементите на списък, може да се използва стандартна функция за печат, която отпечатва съдържанието на списъка.

Пример:

```
numbers = [1, 2, 3, 4, 5]
```

```
people = ["Tom", "Sam", "Bob"]
```

```
print(numbers)          # [1, 2, 3, 4, 5]
```

```
print(people)           # ["Tom", "Sam", "Bob"]
```

Тема 6. Списъци, кортежи и речници

Конструктор на списък

Списъкът може да приеме набор от стойности, въз основа на които се създава самият списък.

Пример:

```
numbers1 = [1, 2, 3, 4, 5]
numbers2 = list(numbers1)
print(numbers2)          # [1, 2, 3, 4, 5]
letters = list("Hello")
print(letters)           # ['H', 'e', 'l', 'l', 'o']
```

Ако трябва да се създаде списък, в който една и съща стойност се повтаря няколко пъти, се използва символа звездичка *, като да се приложи операцията за умножение към вече съществуващ списък.

Пример:

```
numbers = [5] * 6          # 6 пъти се повтаря числото 5
print(numbers)             # [5, 5, 5, 5, 5, 5]

people = ["Tom"] * 3       # 3 пъти се повтаря "Tom"
print(people)              # ["Tom", "Tom", "Tom"]

students = ["Bob", "Sam"] * 2 # 2 пъти се повтаря "Bob", "Sam"
print(students)            # ["Bob", "Sam", "Bob", "Sam"]
```

Тема 6. Списъци, кортежи и речници

Достъп до елементи от списъка

За достъп до елементите на списък трябва да се използват индекси, които представляват номера на елемента в списъка. Индексите започват от нула. Т.е. първият елемент ще има индекс 0, вторият елемент ще има индекс 1 и т.н. За достъп до елементи от края може да се използват отрицателни индекси, започващи от -1. Т.е. последният елемент ще има индекс -1, предпоследният ще има индекс -2 и т.н.

Пример:

```
people = ["Tom", "Sam", "Bob"]  
# получаване на елементи от началото на списъка  
print(people[0]) # Tom  
print(people[1]) # Sam  
print(people[2]) # Bob
```

Пример:

```
# получаване на елементи от края на списъка  
print(people[-2]) # Sam  
print(people[-1]) # Bob  
print(people[-3]) # Tom
```

Пример:

За да се промени елемент от списък, достатъчно е да му се зададе нова стойност:

```
people = ["Tom", "Sam", "Bob"]  
people[1] = "Mike" # промяна на втория елемент  
print(people[1])   # Mike  
print(people)      # ["Tom", "Mike", "Bob"]
```

Тема 6. Списъци, кортежи и речници

Разлагане на списък

Python позволява да се декомпозира списък на отделни елементи.

Пример:

```
people = ["Tom", "Bob", "Sam"]
tom, bob, sam = people
print(tom)    # Tom
print(bob)    # Bob
print(sam)    # Sam
```

В този случай променливите `tom`, `bob` и `sam` са последователно присвоени елементи от списъка `people`. Обаче, трябва да се има предвид, че броят на променливите трябва да е равен на броя на елементите от присвоения списък.

Итерация върху елементи

Може да се използва, както цикъл `for`, така и цикъл `while`, за да се преглеждат елементите.

Итерация с `for` цикъл:

```
people = ["Tom", "Sam", "Bob"]
for person in people:
    print(person)
```

Тук списъкът с хора ще бъде повторен и всеки елемент ще бъде поставен в променливата `person`.

Итерацията може да се направи и с цикъл `while`:

```
people = ["Tom", "Sam", "Bob"]
i = 0
while i < len(people):
    print(people[i])    # използване на индекс, за получаване на елемент
    i += 1
```

Тема 6. Списъци, кортежи и речници

В цикъла се използва функцията `len()`, чрез, която се получава дължината на списъка. С помощта на брояча `i` се отпечатва елемент по елемент, докато стойността на брояча е равна на дължината на списъка.

Сравнение на списъци

Два списъка се считат за равни, ако съдържат един и същ набор от елементи:

```
numbers1 = [1, 2, 3, 4, 5]
numbers2 = list([1, 2, 3, 4, 5])
if numbers1 == numbers2:
    print("numbers1 equal to numbers2")
else:
    print("numbers1 is not equal to numbers2")
```

В този случай и двата списъка ще бъдат равни.

Изброяване на методи и функции

Списъците имат редица методи за манипулиране на елементи. Някои от тях:

- **append(item):** добавя елемент в края на списъка;
- **insert(index, item):** добавя елемент към списъка с индекс;
- **extend(items):** добавя набор от елементи в края на списъка;
- **remove(item):** премахва елемента item. Премахва се само първата поява на елемента. Ако елементът не бъде намерен, се хвърля изключение ValueError;
- **clear():** премахва всички елементи от списъка;
- **index(item):** Връща индекса на елемента. Ако елементът не бъде намерен, хвърля изключение ValueError;
- **pop([index]):** премахва и връща елемента в index. Ако не се предава индекс, тогава просто премахва последния елемент;
- **count(item):** връща броя на появата на елемент в списъка;
- **sort([key]):** сортира елементите. Сортира във възходящ ред по подразбиране. Но с помощта на ключовия параметър може да се предаде функция за сортиране;
- **reverse():** обръща всички елементи в списък;
- **copy():** копира списъка.

В допълнение, Python предоставя редица вградени функции за работа със списъци:

- `len(list)`: връща дължината на списъка;
- `sorted(list, [key])`: връща сортиран списък;
- `min(list)`: връща най-малкия елемент от списъка;
- `max(list)`: връща най-големия елемент от списъка.

Добавяне и премахване на елементи

Методите се използват за добавяне на елемент: `append()`, `extend` и `insert`, а за премахване на елемент: `remove()`, `pop()`, `clear()`.

Използване на методи:

```
people = ["Tom", "Bob"]
```

добавяне в края на списъка

```
people.append("Alice") # ["Tom", "Bob", "Alice"]
```

добавяне на втора позиция

```
people.insert(1, "Bill") # ["Tom", "Bill", "Bob", "Alice"]
```

добавяне на набор от елементи ["Mike", "Sam"]

```
people.extend(["Mike", "Sam"]) # ["Tom", "Bill", "Bob", "Alice", "Mike", "Sam"]
```

вземане на индекса на елемента

```
index_of_tom = people.index("Tom")
```

Тема 6. Списъци, кортежи и речници

премахване на елемент по индекс

```
removed_item = people.pop(index_of_tom) # ["Bill", "Bob", "Alice", "Mike", "Sam"]
```

премахване на последния елемент

```
last_item = people.pop() # ["Bill", "Bob", "Alice", "Mike"]
```

премахване на елемент "Alice"

```
people.remove("Alice") # ["Bill", "Bob", "Mike"]
```

```
print(people) # ["Bill", "Bob", "Mike"]
```

премахване на всички елементи

```
people.clear()
```

```
print(people) # []
```

main.py

```
1 people = ["Tom", "Bob"]
2 # добавяне в края на списъка
3 people.append("Alice") # ["Tom", "Bob", "Alice"]
4 # добавяне на втора позиция
5 people.insert(1, "Bill") # ["Tom", "Bill", "Bob", "Alice"]
6 # добавяне на набор от елементи ["Mike", "Sam"]
7 people.extend(["Mike", "Sam"]) # ["Tom", "Bill", "Bob", "Alice", "Mike",
  "Sam"]
8 # вземане на индекса на елемента
9 index_of_tom = people.index("Tom")
10 # премахване на елемент по индекс
11 removed_item = people.pop(index_of_tom) # ["Bill", "Bob", "Alice", "Mike",
  "Sam"]
12 # премахване на последния елемент
13 last_item = people.pop() # ["Bill", "Bob", "Alice", "Mike"]
14 # премахване на елемент "Alice"
15 people.remove("Alice") # ["Bill", "Bob", "Mike"]
16 print(people) # ["Bill", "Bob", "Mike"]
17 # премахване на всички елементи
18 people.clear()
19 print(people) # []
```



Run

Shell

```
['Bill', 'Bob', 'Mike']
[]
> |
```


Тема 6. Списъци, кортежи и речници

Проверка дали съществува елемент

Ако конкретен елемент не бъде намерен, тогава методите `remove` и `index` предизвикват изключение. За да се избегне тази ситуация, преди да работите с елемент, може да проверите за присъствието му, като използвате ключовата дума `in`:

```
people = ["Tom", "Bob", "Alice", "Sam"]
```

```
if "Alice" in people:
```

```
    people.remove("Alice")
```

```
print(people)
```

main.py

```
1 people = ["Tom", "Bob", "Alice", "Sam"]
2 if "Alice" in people:
3     people.remove("Alice")
4 print(people)
```



Run

Shell

```
['Tom', 'Bob', 'Sam']
```

```
> |
```

Изразът `if "Alice" in people` връща `True`, ако елементът `"Alice"` присъства в списъка с `people`. Следователно, конструкцията `if "Alice" in people` може да изпълни следващия блок от инструкции, в зависимост от присъствието на елемента в списъка.

Изтриване с `del`

Python също така поддържа друг начин за премахване на елементи от списък, като използва изразът `del`. Елементът или наборът от елементи, които трябва да бъдат премахнати, се предава като параметър на този оператор:

main.py



Run

Shell

```
1 people = ["Tom", "Bob", "Alice", "Sam", "Bill", "Kate", "Mike"]
2 del people[1]          # премахване на втория елемент
3 print(people)          # ["Tom", "Alice", "Sam", "Bill", "Kate", "Mike"]
4 del people[:3]         # премахване на 4-ят елемент, без да се включва (премахване на
                          # всички 4-ри елемента)
5 print(people)          # ["Bill", "Kate", "Mike"]
6 del people[1:]          # премахване от втория елемент
7 print(people)          # ["Bill"]
```

```
['Tom', 'Alice', 'Sam', 'Bill', 'Kate', 'Mike']
['Bill', 'Kate', 'Mike']
['Bill']
> |
```

Брой на срещанията в списък

Ако трябва да разберете колко пъти даден елемент присъства в списъка, може да използвате метода `count()`:

```
people = ["Tom", "Bob", "Alice", "Tom", "Bill", "Tom"]
people_count = people.count("Tom")
print(people_count)          # 3
```

Сортиране

Методът `sort()` се използва за сортиране във възходящ ред:

```
people = ["Tom", "Bob", "Alice", "Sam", "Bill"]
people.sort()
print(people)                # ["Alice", "Bill", "Bob", "Sam", "Tom"]
```

Ако е необходимо да сортирате данните в обратен ред, тогава може да приложите метода след сортиране `reverse()`:

```
people = ["Tom", "Bob", "Alice", "Sam", "Bill"]
people.sort()
people.reverse()
print(people)                # ["Tom", "Sam", "Bob", "Bill", "Alice"]
```

Тема 6. Списъци, кортежи и речници

Сортирането всъщност сравнява два обекта и кой от тях е "по-малко" се поставя пред този, който е "по-голям". Понятията "повече" и "по-малко" са доста произволни. И ако за числата всичко е просто - числата са подредени във възходящ ред, то за низове и други обекти ситуацията е по-сложна. По-специално, **низовете се оценяват по първите символи.** Ако първите знаци са равни, се оценяват вторите и т.н. **При което цифров знак се счита за "по-малко" от буквен главен знак, а главен знак се счита за по-малък от символ с малки букви.** По този начин, ако списъкът комбинира низове с главни и малки букви, тогава могат да получат не съвсем правилни резултати, тъй като за нас низът "bob" трябва да идва преди низа "Tom". И за да се промени стандартното поведение на сортиране, може да се предаде `sort()` функцията, като параметър на метода:

```
people = ["Tom", "bob", "alice", "Sam", "Bill"]
```

<code>people.sort()</code>	<code># стандартна сортировка</code>
<code>print(people)</code>	<code># ["Bill", "Sam", "Tom", "alice", "bob"]</code>
<code>people.sort(key=str.lower)</code>	<code># сортировка без счетоводен регистър</code>
<code>print(people)</code>	<code># ["alice", "Bill", "bob", "Sam", "Tom"]</code>

Тема 6. Списъци, кортежи и речници

В допълнение към метода за сортиране, може да се използва сортираната вградена функция , която има две форми:

- **sorted(list):** сортира списък
- **sorted(list, key):** сортира списъка със списък, като прилага ключовата функция към елементите

```
people = ["Tom", "bob", "alice", "Sam", "Bill"]
sorted_people = sorted(people, key=str.lower)
print(sorted_people)           # ["alice", "Bill", "bob", "Sam", "Tom"]
```

Когато се използва тази функция, трябва да се има предвид, че тази функция не променя сортирания списък, но поставя всички сортирани елементи в нов списък, който се връща като резултат.

Минимални и максимални стойности

Вградените функции на Python `min()` и `max()` позволява търсене на минималните и максималните стойности:

```
numbers = [9, 21, 12, 1, 3, 15, 18]
print(min(numbers))           # 1
print(max(numbers))           # 21
```

Копиране на списъци

Когато копирате списъци, трябва да се има предвид, че списъците представляват променлив тип, така че ако и двете променливи сочат към един и същ списък, тогава промяната на една променлива ще повлияе на друга променлива:

```
people1 = ["Tom", "Bob", "Alice"]
people2 = people1
people2.append("Sam")           # добавете елемент към втория списък
                                # people1 и people2 сочат към един и същ списък
print(people1)                 # ["Tom", "Bob", "Alice", "Sam"]
print(people2)                 # ["Tom", "Bob", "Alice", "Sam"]
```

Това е така нареченото "**плитко копие**". И като правило такова поведение е нежелателно. За да копирате елементите, но в същото време променливите да сочат към различни списъци, трябва да извършите дълбоко копиране. Можете да използвате метода `copy()` за това:

```
people1 = ["Tom", "Bob", "Alice"]
people2 = people1.copy()       # копирайте елементи от people1 в people2
people2.append("Sam")          # добавете елемент САМО към втория списък
                                # people1 и people2 вече са различни списъци
print(people1)                 # ["Tom", "Bob", "Alice"]
print(people2)                 # ["Tom", "Bob", "Alice", "Sam"]
```

Копиране на част от списък

Ако е необходимо да се копира не целия списък, а само конкретна част от него, тогава може да се използва специален синтаксис, който може да приеме следните форми:

- **list[:end]:** индексът на елемента, към който трябва да се копира списъкът, се предава през крайния параметър
- **list[start:end]:** началният параметър сочи към индекса на елемента, от който да се копират елементите
- **list[start:end:step]:** параметърът step показва стъпката, през която ще бъдат копирани елементите от списъка. По подразбиране този параметър е 1.

```
people = ["Tom", "Bob", "Alice", "Sam", "Tim", "Bill"]
```

```
slice_people1 = people[:3]           # от 0 до 3
print(slice_people1)                 # ["Tom", "Bob", "Alice"]
```

```
slice_people2 = people[1:3]          # от 1 до 3
print(slice_people2)                 # ["Bob", "Alice"]
```

```
slice_people3 = people[1:6:2]        # от 1 до 6 през 2 стъпки
print(slice_people3)                 # ["Bob", "Sam", "Bill"]
```

Тема 6. Списъци, кортежи и речници

Списъци за свързване

Операторът за добавяне (+) се използва за комбиниране на списъци:

```
people1 = ["Tom", "Bob", "Alice"]
people2 = ["Tom", "Sam", "Tim", "Bill"]
people3 = people1 + people2
print(people3)                # ["Tom", "Bob", "Alice", "Tom", "Sam", "Tim", "Bill"]
```

Списъци със списъци

Списъците, освен стандартни данни като низове, числа, могат да съдържат и други списъци. Такива списъци могат да бъдат свързани с таблици, където вложените списъци действат като редове. Например:

```
people = [
    ["Tom", 29],
    ["Alice", 33],
    ["Bob", 27]
]

print(people[0])              # ["Tom", 29]
print(people[0][0])           # Tom
print(people[0][1])           # 29
```

Тема 6. Списъци, кортежи и речници

За обръщение към елемент от вложен списък, трябва да се използват двойка индекси: `people[0][1]` - достъп до втория елемент от първия вложен списък. Добавянето, изтриването и промяната на общия списък, както и на вложените списъци, е подобно на начина, по който се прави с обикновени (едномерни) списъци:

```
people = [  
    ["Tom", 29],  
    ["Alice", 33],  
    ["Bob", 27]  
]
```

създаване на вложен списък

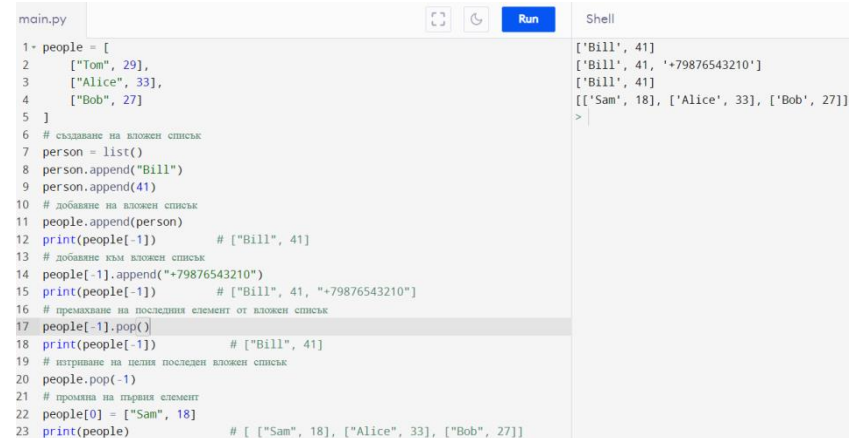
```
person = list()  
person.append("Bill")  
person.append(41)
```

добавяне на вложен списък

```
people.append(person)  
print(people[-1])    # ["Bill", 41]
```

добавяне към вложен списък

```
people[-1].append("+79876543210")  
print(people[-1])    # ["Bill", 41, "+79876543210"]
```



```
main.py  
1 people = [  
2     ["Tom", 29],  
3     ["Alice", 33],  
4     ["Bob", 27]  
5 ]  
6 # създаване на вложен списък  
7 person = list()  
8 person.append("Bill")  
9 person.append(41)  
10 # добавяне на вложен списък  
11 people.append(person)  
12 print(people[-1])    # ["Bill", 41]  
13 # добавяне към вложен списък  
14 people[-1].append("+79876543210")  
15 print(people[-1])    # ["Bill", 41, "+79876543210"]  
16 # премахване на последния елемент от вложен списък  
17 people[-1].pop()  
18 print(people[-1])    # ["Bill", 41]  
19 # изтриване на целия последен вложен списък  
20 people.pop(-1)  
21 # промяна на първия елемент  
22 people[0] = ["Sam", 18]  
23 print(people)        # [ ["Sam", 18], ["Alice", 33], ["Bob", 27]]
```

```
Shell  
['Bill', 41]  
['Bill', 41, '+79876543210']  
['Bill', 41]  
[['Sam', 18], ['Alice', 33], ['Bob', 27]]  
>
```

премахване на последния елемент от вложен списък

```
people[-1].pop()  
print(people[-1])    # ["Bill", 41]
```

изтриване на целия последен вложен списък

```
people.pop(-1)
```

промяна на първия елемент

```
people[0] = ["Sam", 18]  
print(people)        # [ ["Sam", 18], ["Alice", 33],  
["Bob", 27]]
```


Тема 6. Списъци, кортежи и речници

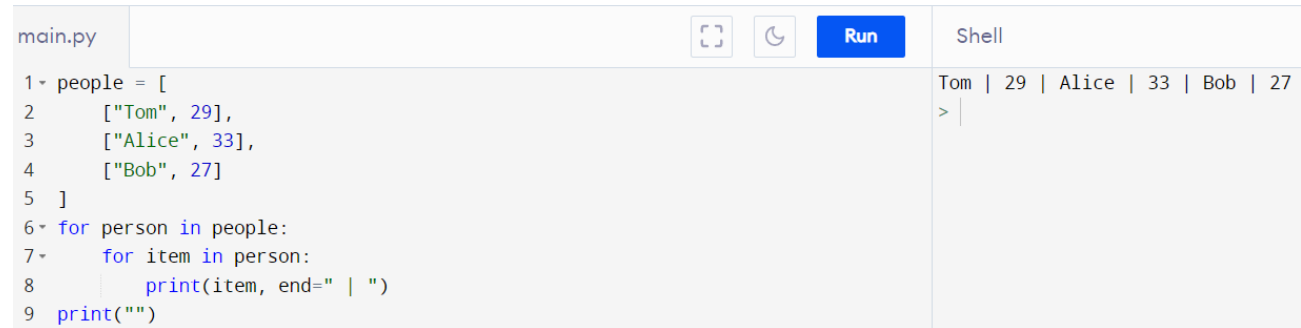
Итерация върху вложени списъци:

```
people = [  
    ["Tom", 29],  
    ["Alice", 33],  
    ["Bob", 27]  
]
```

```
for person in people:  
    for item in person:  
        print(item, end=" | ")  
print("")
```

Изход в конзолата:

Tom | 29 | Alice | 33 | Bob | 27 |



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```
1- people = [  
2     ["Tom", 29],  
3     ["Alice", 33],  
4     ["Bob", 27]  
5 ]  
6- for person in people:  
7     for item in person:  
8         print(item, end=" | ")  
9 print("")
```

On the right side, there is a 'Shell' window showing the output of the script:

```
Tom | 29 | Alice | 33 | Bob | 27  
>
```

Кортежи

Кортежът представлява **последователност от елементи, подобно на списък, с изключение на това, че кортежът е неизменяем тип. Следователно не може да се добавят или премахват елементи в кортеж, така, че да се променя.** За създаване на кортеж се използват скоби, в които се поставят стойностите му, разделени със запетаи:

```
tom = ("Tom", 23)
```

```
print(tom)
```

```
# ("Tom", 23)
```

Тема 6. Списъци, кортежи и речници

Също така, за да се дефинира кортеж, може просто да се изброят стойностите, разделени със запетай, без да се използват скоби:

```
tom = "Tom", 23  
print(tom)          # ("Tom", 23)
```

Ако кортежът се състои от един елемент, тогава след единствения елемент от кортежа трябва да се постави запетая:

```
tom = ("Tom",)
```

За да се създаде кортеж от друг набор от елементи, като списък, може да се предаде списъка на функцията `tuple()`, която ще върне кортеж:

```
data = ["Tom", 37, "Google"]  
tom = tuple(data)  
print(tom)          # ("Tom", 37, "Google")
```

Може да се получи дължината на кортежа с помощта на вградената функция `len()`:

```
tom = ("Tom", 37, "Google")  
print(len(tom))     # 3
```

Достъп до елементи от кортеж

Достъпът до елементи в кортеж се осъществява по същия начин, както в списък, чрез индекс. Индексирането също започва от нула при получаване на елементи от началото на списъка и от -1 при получаване на елементи от края на списъка:

```
tom = ("Tom", 37, "Google", "software developer")
print(tom[0])    # Tom
print(tom[1])    # 37
print(tom[-1])   # software developer
```

Но тъй като кортежът е неизменяем тип, не може да се променят елементите му, т.е. записът:

```
tom[1] = "Tim,, (няма да работи.)
```

Ако е необходимо, може да се декомпозира кортежа на отделни променливи:

```
name, age, company, position = ("Tom", 37, "Google", "software developer")
print(name)    # Tom
print(age)     # 37
print(position) # software developer
print(company) # Google
```

Получаване на подкортеж

Както при списъците, може да се получи част от кортеж като друг кортеж:

```
tom = ("Tom", 37, "Google", "software developer")
```

```
# подкортеж от 1-ви до 3-ти елемент (без да се включва 1-ви и 3-ти елемент)  
print(tom[1:3])    # (37, "Google")
```

```
# подкортеж от 0 до 3-ти елемент (без да се включва 3-ти елемент)  
print(tom[:3])    # ("Tom", 37, "Google")
```

```
# подкортеж от 1 до последния елемент (не се включва 0-ят елемент)  
print(tom[1:])    # (37, "Google", "software developer")
```

Кортеж като параметър и резултат от функции

Особено удобно е да се използват кортежи, когато трябва да се върнат няколко стойности от функция наведнъж. Когато функция връща множество стойности, тя всъщност връща кортеж:

```
def get_user():  
    name = "Tom"  
    age = 22  
    company = "Google"  
    return name, age, company  
  
user = get_user()  
print(user)    # ("Tom", 22, "Google")
```

Тема 6. Списъци, кортежи и речници

Когато предавате кортеж към функция с помощта на оператора *, той може да бъде разложен на отделни стойности, които се предават на параметрите на функцията:

```
def print_person(name, age, company):  
    print(f"Name: {name} Age: {age} Company: {company}")  
  
tom = ("Tom", 22)  
print_person(*tom, "Microsoft")    # Name: Tom Age: 22 Company: Microsoft  
  
bob = ("Bob", 41, "Apple")  
print_person(*bob)                 # Name: Bob Age: 41 Company: Apple
```

Итерация върху кортежи

Може да използвате стандартните цикли for и while, за да итерирате над кортеж. С **цикъл for**:

```
tom = ("Tom", 22, "Google")  
for item in tom:  
    print(item)
```

С цикъл **while**:

```
tom = ("Tom", 22, "Google")
i = 0
while i < len(tom):
    print(tom[i])
    i += 1
```

Проверка дали съществува стойност

Що се отнася до списък, използващ израз елемент **in** кортеж, може да се провери наличието на елемент в кортеж:

```
user = ("Tom", 22, "Google")
name = "Tom"
if name in user:
    print("Името на потребителя е Tom")
else:
    print("Потребителят има различно име")
```

Обхвати

Диапазоните или диапазонът представляват неизменяем последователен набор от числа. За да се създадат диапазони, използвайте `range`, който има следните форми:

- `диапазон(стоп)`: връща всички цели числа от 0 до стоп
- `диапазон(старт, стоп)`: връща всички цели числа между начало (включително) и стоп (с изключение).
- `диапазон(начало, спиране, стъпка)`: връща цели числа между начало (включително) и стоп (без да се включва), увеличено със стойността на стъпката

Примери за извикване на функция за диапазон:

```
range(5)           # 0, 1, 2, 3, 4
range(1, 5)        # 1, 2, 3, 4
range(2, 10, 2)    # 2, 4, 6, 8
range(10, 2, -2)   # 10 8 6 4
```

Диапазоните се използват най-често в циклите `for`. Например, нека се отпечатаат всички числа от 0 до 4 последователно:

```
for i in range(5):
    print(i, end=" ")
# Изход в конзолата
# 0, 1, 2, 3, 4
```


Тема 6. Списъци, кортежи и речници

Друг пример е да се покаже таблицата за умножение:

```
for i in range(1, 10):  
    for j in range(1, 10):  
        print(i * j, end="\t")  
    print("\n")
```

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Ако е необходим последователен списък с числа, тогава е удобно да се използва функцията за диапазон, за да бъде създаден:

```
numbers = list(range(10))  
print(numbers)    # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
numbers = list(range(2, 10))  
print(numbers)    # [2, 3, 4, 5, 6, 7, 8, 9]  
numbers = list(range(10, 2, -2))  
print(numbers)    # [10, 8, 6, 4]
```

The screenshot shows a code editor with the following Python code in `main.py`:

```
1- for i in range(1, 10):  
2-     for j in range(1, 10):  
3-         print(i * j, end="\t")  
4-     print("\n")
```

The output in the Shell window is the 9x9 multiplication table:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Предимството на диапазоните пред стандартните списъци и кортежи е, че диапазонът винаги ще заема едно и също малко количество памет, независимо какъв набор от числа представлява диапазонът. В действителност диапазонът съхранява само началните, крайните и нарастващите стойности.

Речници

Речникът в Python съхранява колекция от елементи, където всеки елемент има уникален ключ и някаква стойност, свързана с него.

Дефиницията на речника, синтаксис:

```
dictionary = { ключ1:значение1, ключ2:значение2, ....}
```

Къдравите скоби дефинират последователност от елементи, разделена със запетая, където за всеки елемент първо се посочва ключът и стойността му се разделя с двоеточие.

Нека се дефинира речник:

```
users = {1: "Tom", 2: "Bob", 3: "Bill"}
```

Потребителският речник използва числа като ключове и низове като стойности. Т.е. елементът с ключ 1 има стойността "Tom", елементът с ключ 2 има стойността "Bob" и т.н.

Пример:

```
emails = {"tom@gmail.com": "Tom", "bob@gmail.com": "Bob", "sam@gmail.com": "Sam"}
```

Речникът за имейли използва низове като ключове - потребителски имейл адреси и низове - потребителски имена като стойности.

Но не е задължително ключовете и низовете да са от един и същи тип. Те могат да представляват различни видове:

```
objects = {1: "Tom", "2": True, 3: 100.6}
```

Тема 6. Списъци, кортежи и речници

Може също да се дефинира празен речник без елементи:

```
objects = {}
```

или така:

```
objects = dict()
```

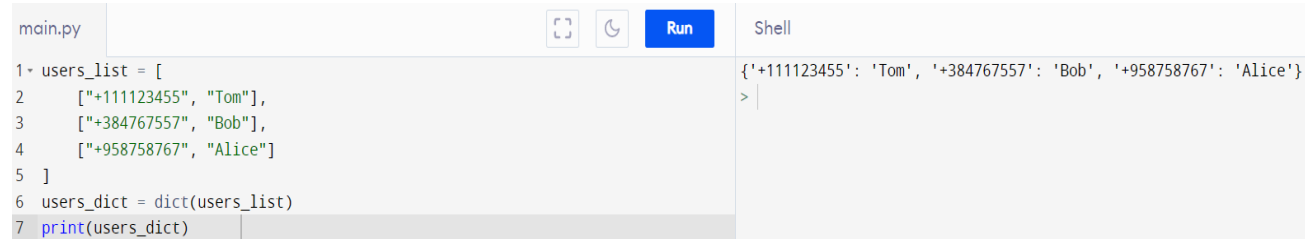
Преобразуване на списъци и кортежи в речник

Въпреки че речникът и списъкът са структурно различни типове, все още е възможно някои видове списъци да бъдат преобразувани в речник с помощта на вградената функция `dict()`. За да направите това, списъкът трябва да съхранява набор от вложени списъци. Всеки вложен списък трябва да се състои от два елемента - когато се преобразува в речник, първият елемент става ключ, а вторият става стойността:

```
users_list = [  
    ["+111123455", "Tom"],  
    ["+384767557", "Bob"],  
    ["+958758767", "Alice"]  
]
```

```
users_dict = dict(users_list)
```

```
print(users_dict)      # {"+111123455": "Tom", "+384767557": "Bob", "+958758767":  
"Alice"}
```



The screenshot shows a code editor with a file named `main.py`. The code defines a list `users_list` containing three sub-lists, each with a phone number and a name. It then uses `dict(users_list)` to create a dictionary `users_dict` and prints it. The output in the shell is a dictionary with the same key-value pairs.

```
main.py
1 users_list = [
2     ["+111123455", "Tom"],
3     ["+384767557", "Bob"],
4     ["+958758767", "Alice"]
5 ]
6 users_dict = dict(users_list)
7 print(users_dict)
```

```
Shell
{'+111123455': 'Tom', '+384767557': 'Bob', '+958758767': 'Alice'}
>
```

Тема 6. Списъци, кортежи и речници

По същия начин, два кортежа могат да бъдат преобразувани в речник, който от своя страна съдържа два кортежа:

```
users_tuple = (  
    "+111123455", "Tom"),  
    "+384767557", "Bob"),  
    "+958758767", "Alice")  
)  
users_dict = dict(users_tuple)  
print(users_dict)
```

Получаване и промяна на елементи

За препратка към елементите на речника, след неговото име, ключът на елемента е посочен в квадратни скоби:

```
dictionary[ключ]
```

Тема 6. Списъци, кортежи и речници

Например, за да се получат и промени в елементите в речник:

```
users = {  
    "+11111111": "Tom",  
    "+33333333": "Bob",  
    "+55555555": "Alice"  
}  
  
# получаване на елемент с ключ "+11111111"  
print(users["+11111111"])    # Tom  
  
# задаване на стойността на елемент с ключ "+33333333"  
users["+33333333"] = "Bob Smith"  
print(users["+33333333"])    # Bob Smith
```

Ако при задаване на стойността на елемент с такъв ключ няма елемент в речника, тогава той ще бъде добавен:

```
users["+44444444"] = "Sam"
```

Но ако се опита да се получи стойност с ключ, който не е в речника, тогава Python ще изведе `KeyError`:

```
user = users["+44444444"]    # KeyError
```

Тема 6. Списъци, кортежи и речници

За да се предотврати тази ситуация, преди да се осъществи достъп до елемента, може да се провери за наличието на ключ в речника, като се използва ключа на израза в dictionary. Ако ключът е в речника, тогава този израз връща True:

```
key = "+44444444"
if key in users:
    user = users[key]
    print(user)
else:
    print("Елементът не е открит")
```

Може също да се използва метода `get`, за да се получат елементи, има две форми:

- **`get(key)`**: връща елемента с ключа от речника. Ако няма елемент с този ключ, той връща `None`.
- **`get(key, default)`**: връща елемента с ключа от речника. Ако няма елемент с такъв ключ, той връща стойността по подразбиране

```
users = {
    "+11111111": "Tom",
    "+33333333": "Bob",
    "+55555555": "Alice"
}
```

Тема 6. Списъци, кортежи и речници

```
user1 = users.get("+55555555")
print(user1)  # Alice
user2 = users.get("+33333333", "Unknown user")
print(user2)  # Bob
user3 = users.get("+44444444", "Unknown user")
print(user3)  # Unknown user
```

Премахване

Инструкцията `del` се използва за премахване на елемент по ключ:

```
users = {
    "+11111111": "Tom",
    "+33333333": "Bob",
    "+55555555": "Alice"
}
del users["+55555555"]
print(users)  # { "+11111111": "Tom", "+33333333": "Bob" }
```

Но трябва да се има предвид, че ако такъв ключ не е в речника, тогава ще бъде изведено изключение `KeyError`. Ето защо, отново, преди изтриването е желателно да се провери наличието на елемент с даден ключ.

Тема 6. Списъци, кортежи и речници

```
users = {  
    "+11111111": "Tom",  
    "+33333333": "Bob",  
    "+55555555": "Alice"  
}  
key = "+55555555"  
if key in users:  
    del users[key]  
    print(f"Елемент с ключ {key} е премахнат")  
else:  
    print("Елемента не е открит")
```

Друг метод за премахване е методът pop(). Има също две форми:

- **pop(key):** премахва елемента с ключ и връща премахнатия елемент. Ако няма елемент с дадения ключ, тогава се хвърля изключение `KeyError`.
- **pop(key, default):** премахва елемента с ключ и връща премахнатия елемент. Ако няма елемент с дадения ключ, тогава се връща по подразбиране.

Тема 6. Списъци, кортежи и речници

```
users = {  
    "+11111111": "Tom",  
    "+33333333": "Bob",  
    "+55555555": "Alice"  
}  
key = "+55555555"  
user = users.pop(key)  
print(user)    # Alice  
user = users.pop("+4444444", "Unknown user")  
print(user)    # Unknown user
```

Ако трябва да се премахнат всички елементи, тогава в този случай може да се използва метода `clear()`:

```
users.clear()
```

Копиране и сливане на речници

Методът `copy()` копира съдържанието на речник, връщайки нов речник:

```
users = {"+1111111": "Tom", "+3333333": "Bob", "+5555555": "Alice"}  
students = users.copy()  
print(students)    # {"+1111111": "Tom", "+3333333": "Bob", "+5555555": "Alice"}
```

Тема 6. Списъци, кортежи и речници

Методът `update()` обединява два речника:

```
users = {"+1111111": "Tom", "+3333333": "Bob"}
```

```
users2 = {"+2222222": "Sam", "+6666666": "Kate"}
```

```
users.update(users2)
```

```
print(users)  # {"+1111111": "Tom", "+3333333": "Bob", "+2222222": "Sam", "+6666666":  
"Kate"}
```

```
print(users2) # {"+2222222": "Sam", "+6666666": "Kate"}
```

В този случай речникът на `users2` остава непроменен. Променя се само потребителският речник, към който се добавят елементи от друг речник. Но ако е необходимо и двата изходни речника да останат непроменени и резултатът от обединението е някакъв трети речник, тогава може първо да се копира един речник в друг:

```
users3 = users.copy()
```

```
users3.update(users2)
```

Итерация на речник

Може да използвате цикъл `for`, за да преминете през речник:

```
users = {  
    "+11111111": "Tom",  
    "+33333333": "Bob",  
    "+55555555": "Alice"  
}
```

`for key in users:`

```
    print(f"Phone: {key} User: {users[key]} ")
```

Когато се итерира през елементите, се получава ключа на текущия елемент и може да се използва, за да се получи самия елемент.

Друг начин за повторение на елементи е да се използва метода `items()`:

```
users = {  
    "+11111111": "Tom",  
    "+33333333": "Bob",  
    "+55555555": "Alice"  
}
```

`for key, value in users.items():`

```
    print(f"Phone: {key} User: {value} ")
```

Тема 6. Списъци, кортежи и речници

Методът `items()` връща набор от кортежи. Всеки кортеж съдържа ключа и стойността на елемента, които при повторение можем веднага да влезем в променливите с ключ и стойност.

Има и отделни опции за итерация по ключове и итерация върху стойности. За да се премине през ключовете, може да се извика метода `keys()` на речника:

```
for key in users.keys():  
    print(key)
```

Вярно е, че този метод на изброяване няма смисъл, тъй като без да се извика метода `keys()`, може да се изброят ключовете, както е показано по-горе.

За да се повторят само стойностите, може да се извика метода `values()` на речника :

```
for value in users.values():  
    print(value)
```

Сложни речници

В допълнение към най-простите обекти като числа и низове, в речниците могат да съхраняват и по-сложни обекти - същите списъци, кортежи или други речници:

```
users = {  
    "Tom": {  
        "phone": "+971478745",  
        "email": "tom12@gmail.com"  
    },  
    "Bob": {  
        "phone": "+876390444",  
        "email": "bob@gmail.com",  
        "skype": "bob123"  
    }  
}
```

В този случай стойността на всеки елемент от речника от своя страна представлява отделен речник.

За достъп до елементите на вложен речник, съответно, трябва да се използват два ключа:

Тема 6. Списъци, кортежи и речници

```
old_email = users["Tom"]["email"]
users["Tom"]["email"] = "supertom@gmail.com"
print(users["Tom"])    # { phone": "+971478745", "email": "supertom@gmail.com }
```

Но ако се прави опит да се получи стойност чрез ключ, който не е в речника, Python ще изведе изключение `KeyError`:

```
tom_skype = users["Tom"]["skype"]  # KeyError
```

За да се избегне грешка, може да се провери за наличието на ключ в речника:

```
key = "skype"
if key in users["Tom"]:
    print(users["Tom"]["skype"])
else:
    print("skype is not found")
```

Във всички останали отношения работата със сложни и вложени речници е подобна на работата с обикновени речници.

Комплекти

Наборът съдържа само уникални елементи. За дефиниране на набор се използват къдрави скоби, в които са изброени елементите:

```
users = {"Tom", "Bob", "Alice", "Tom"}  
print(users)  # {"Alice", "Bob", "Tom"}
```

Забележете, че въпреки че функцията за печат извежда елемента "Tom" веднъж, дефиницията на набора съдържа този елемент два пъти. Това е така, защото наборът съдържа само уникални стойности.

Също така, за да се дефинира набор, може да се използва функцията `set()`, на която се предава списък или набор от елементи:

```
people = ["Mike", "Bill", "Ted"]  
users = set(people)  
print(users)  # {"Mike", "Bill", "Ted"}
```

Функцията `set` е полезна за създаване на празен набор:

```
users = set()
```

За да получите дължината на набор, използвайте вградената функция `len()`:

```
users = {"Tom", "Bob", "Alice"}  
print(len(users))  # 3
```

Добавяне на елементи

За да добавите един елемент, се извиква методът `add()`:

```
users = set()
users.add("Sam")
print(users)
```

Премахване на елементи

За да се премахне един елемент, се извиква методът `remove()`, предавайки елемента, който трябва да бъде премахнат. Но ако такъв елемент не е в набора, тогава ще се генерира грешка. Следователно, преди да изтриете, трябва да проверите за наличието на елемент, като използвате оператора `in`:

```
users = {"Tom", "Bob", "Alice"}

user = "Tom"
if user in users:
    users.remove(user)
print(users)  # {"Bob", "Alice"}
```


Тема 6. Списъци, кортежи и речници

Може също да се използва и метода `discard()` за премахване, който няма да хвърли изключения, ако елементът липсва:

```
users = {"Tom", "Bob", "Alice"}
```

```
users.discard("Tim")  # елемент "Tim" не съществува, и метода не прави нищо  
print(users)  # {"Tom", "Bob", "Alice"}
```

```
users.discard("Tom")  # елемент "Tom" съществува, и метода премахва елемента  
print(users)  # {"Bob", "Alice"}
```

За да премахнете всички елементи, се извиква методът `clear()`:

```
users.clear()
```

Итерация върху набор

Може да използвате цикъл `for`, за да преглеждате елементите:

```
users = {"Tom", "Bob", "Alice"}
```

```
for user in users:  
    print(user)
```

Тема 6. Списъци, кортежи и речници

При повторение всеки елемент се поставя в потребителската променлива.

Задайте операции

Използвайки метода `copy()`, може да копирате съдържанието на един набор в друга променлива:

```
users = {"Tom", "Bob", "Alice"}
students = users.copy()
print(students)  # {"Tom", "Bob", "Alice"}
```

Методът `union()` комбинира два набора и връща нов набор:

```
users = {"Tom", "Bob", "Alice"}
users2 = {"Sam", "Kate", "Bob"}

users3 = users.union(users2)
print(users3)  # {"Bob", "Alice", "Sam", "Kate", "Tom"}
```

Тема 6. Списъци, кортежи и речници

Пресечната точка на множествата позволява да се получат само онези елементи, които са едновременно и в двата набора. Методът `intersection()` изпълнява операция за пресичане на набор и връща нов набор:

```
users = {"Tom", "Bob", "Alice"}  
users2 = {"Sam", "Kate", "Bob"}  
users3 = users.intersection(users2)  
print(users3) # {"Bob"}
```

Вместо метода на пресичане може да се използва операцията за логическо умножение:

```
users = {"Tom", "Bob", "Alice"}  
users2 = {"Sam", "Kate", "Bob"}  
print(users & users2) # {"Bob"}
```

Тема 6. Списъци, кортежи и речници

В този случай ще получим същия резултат. Друга операция - `set different` връща онези елементи, които са в първия набор, но липсват във втория. За да се получи разликата от набори, може да се използва метода `difference` или операцията на изваждане:

```
users = {"Tom", "Bob", "Alice"}
users2 = {"Sam", "Kate", "Bob"}
users3 = users.difference(users2)
print(users3)          # {"Tom", "Alice"}
print(users - users2)  # {"Tom", "Alice"}
```

Отделен вид разлика в набора, симетричната разлика, се произвежда с помощта на метода `symmetric_difference()` . Връща всички елементи от двата набора, с изключение на общите:

```
users = {"Tom", "Bob", "Alice"}
users2 = {"Sam", "Kate", "Bob"}
users3 = users.symmetric_difference(users2)
print(users3)  # {"Tom", "Alice", "Sam", "Kate"}
```

Връзки между множества

Методът `issubset` позволява да се разбере дали текущият набор е подмножество (тоест част) от друг набор:

```
users = {"Tom", "Bob", "Alice"}
superusers = {"Sam", "Tom", "Bob", "Alice", "Greg"}
print(users.issubset(superusers)) # True
print(superusers.issubset(users)) # False
```

Методът `issuperset`, от друга страна, връща `True`, ако текущият набор е супернабор (т.е. съдържа първият) от друг набор:

```
users = {"Tom", "Bob", "Alice"}
superusers = {"Sam", "Tom", "Bob", "Alice", "Greg"}
print(users.issuperset(superusers)) # False
print(superusers.issuperset(users)) # True
```

Замразен комплект

Типът замразен набор е вид набор, който не може да бъде променен. За да се създаде, се използва функцията `frozenset`:

```
users = frozenset({"Tom", "Bob", "Alice"})
```

На функцията `frozenset` се предава набор от елементи - списък, кортеж, друг набор. Не може да се добавят нови елементи към такъв набор, както и да се премахват съществуващи от него. Ето защо замразеният набор поддържа ограничен набор от операции:

- `len(s)`: връща дължината на набора
- `x in s`: връща `True`, ако елемент `x` присъства в набор `s`
- `x not in s`: връща `True`, ако елемент `x` не е в набор `s`
- `s.issubset(t)`: връща `True`, ако `t` съдържа набор от `s`
- `s.issuperset(t)`: връща `True`, ако `t` се съдържа в `s`
- `s.union(t)`: връща обединението на множествата `s` и `t`
- `s.intersection(t)`: връща пресечната точка на `s` и `t`
- `s.difference(t)`: връща разликата на множествата `s` и `t`
- `s.copy()`: връща копие на набора `s`

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009